

UNIVERSIDAD DEL VALLE DE GUATEMALA

Departamento de Ciencias de la Computación

Data Science

Sección 30

Laboratorio 5: Clasificación de tweets usando minería de texto y análisis de sentimiento

Autores:

Daniel Oswaldo Juárez Herrera

Humberto Alexander de la Cruz Chanchavac

Nicolle Alexandra Gordillo

Guatemala, 30 de agosto de 2026

Repositorio: [https:](https://github.com/nicollegordillo/Mineria-de-Textos-y-analisis-de-sentimiento)

[//github.com/nicollegordillo/Mineria-de-Textos-y-analisis-de-sentimiento](https://github.com/nicollegordillo/Mineria-de-Textos-y-analisis-de-sentimiento)

1. Descripción del conjunto de datos

Para este laboratorio trabajamos con el dataset *Natural Language Processing with Disaster Tweets* de Kaggle. El archivo principal que usamos tiene 7,613 filas y 5 columnas principales: `id`, `keyword`, `location`, `text` y `target`. El objetivo principal del proyecto es armar un modelo que prediga si un tweet habla de un desastre real (1) o si es una falsa alarma o broma (0).

Al revisar los datos, notamos que hay un desbalance leve: 4,342 de los tweets (57.03 %) no son desastres y 3,271 (42.97 %) sí lo son. Como la diferencia no es muy grande, decidimos no hacer sobremuestreo, pero prestamos más atención al F1-score que a la exactitud (accuracy) general.

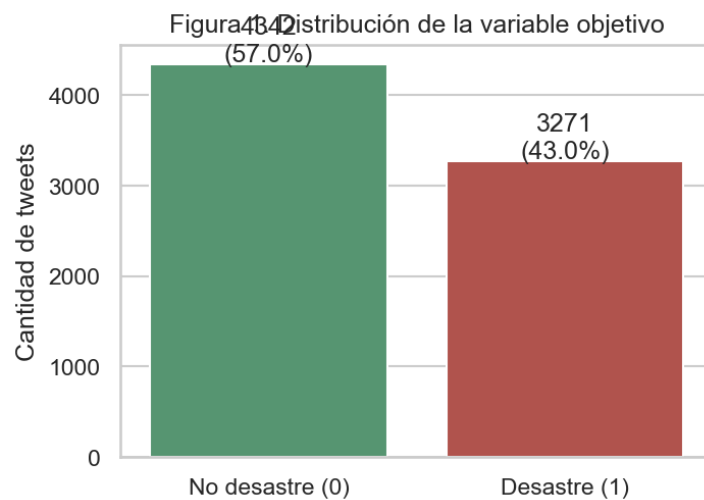


Figura 1: Distribución de la variable objetivo.

7

Un problema con el que nos topamos fue que había 179 tweets con el texto duplicado y, lo más crítico, 18 registros tenían etiquetas contradictorias (el mismo tweet marcado como 0 y 1 a la vez). Esto nos demostró que a veces ni siquiera los humanos se ponen de acuerdo en si algo es un desastre o no, lo que le pone un límite a qué tan perfecto puede ser el modelo.

2. Limpieza y preprocesamiento

Antes de limpiar el texto, nos pusimos a ver qué tipo de cosas traían los tweets. Vimos que cuando se trata de desastres reales, la gente escribe tweets un poco más largos (108 caracteres en promedio contra 96) y suelen poner más links (URLs), lo cual tiene sentido porque normalmente comparten noticias.

Figura 2. Longitud y presencia de URLs según la categoría

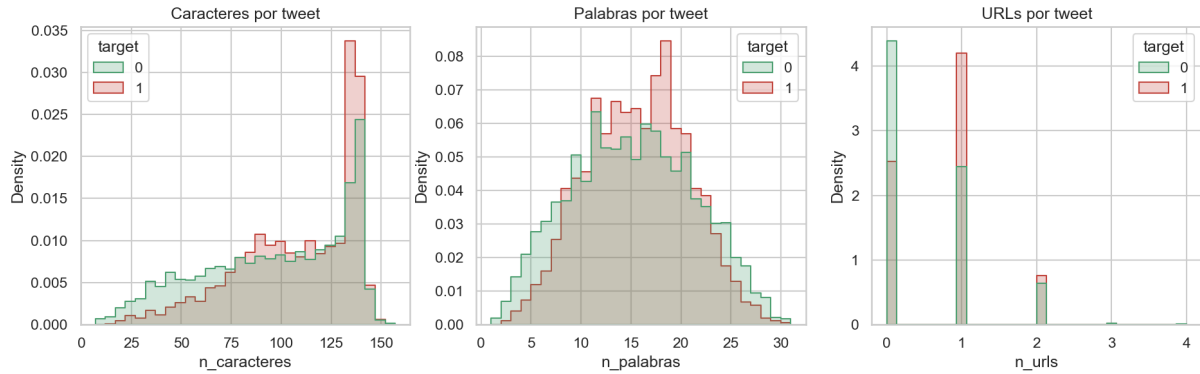


Figura 2: Longitud y presencia de URLs según la categoría.

Para la limpieza, usamos expresiones regulares y la librería NLTK. Los pasos que seguimos fueron:

- Arreglamos los caracteres raros (HTML y de codificación) que dejó el scraping.
- Borramos las URLs y las menciones a usuarios (@) para que el modelo no se memorizara links o cuentas de personas.
- Borramos todos los números, con una sola excepción: el "911". Lo cambiamos por el texto `num911` porque lógicamente es un número importante en emergencias.
- Quitamos las palabras vacías (*stopwords*), pero dejamos las negaciones como *not* o *never* para que el análisis de sentimiento tuviera sentido después.
- Lematizamos todo usando `WordNetLemmatizer` para que palabras como *fires* se tomaran igual que *fire*.

3. Generación de N-gramas y Análisis Exploratorio

Usamos `CountVectorizer` para sacar los unigramas, bigramas y trigramas. Luego, calculamos qué tanto aparecía cada palabra y usamos una métrica llamada log-odds para ver cuáles nos ayudaban más a separar las dos clases.

Figura 3. Top 20 unigramas por categoría

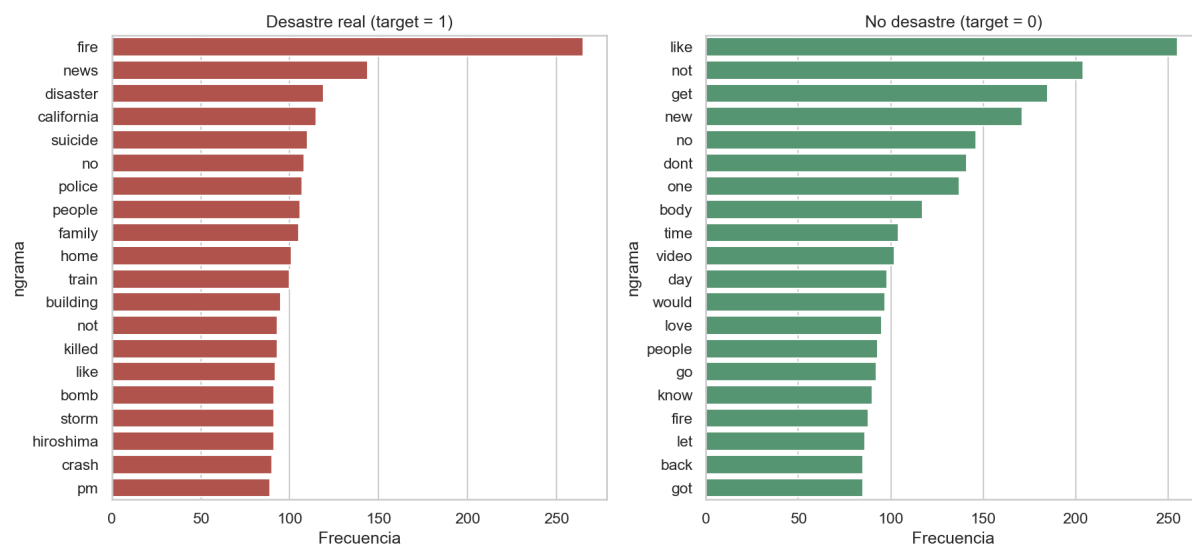


Figura 3: Top 20 unigramas por categoría.

Nos dimos cuenta de que las palabras más comunes no son siempre las mejores. Las que de verdad importan tienen un log-odds muy alto, como *debris*, *derailment* y *legionnaire*. Estas palabras gritan .^{em}ergencia casi cada vez que aparecen.

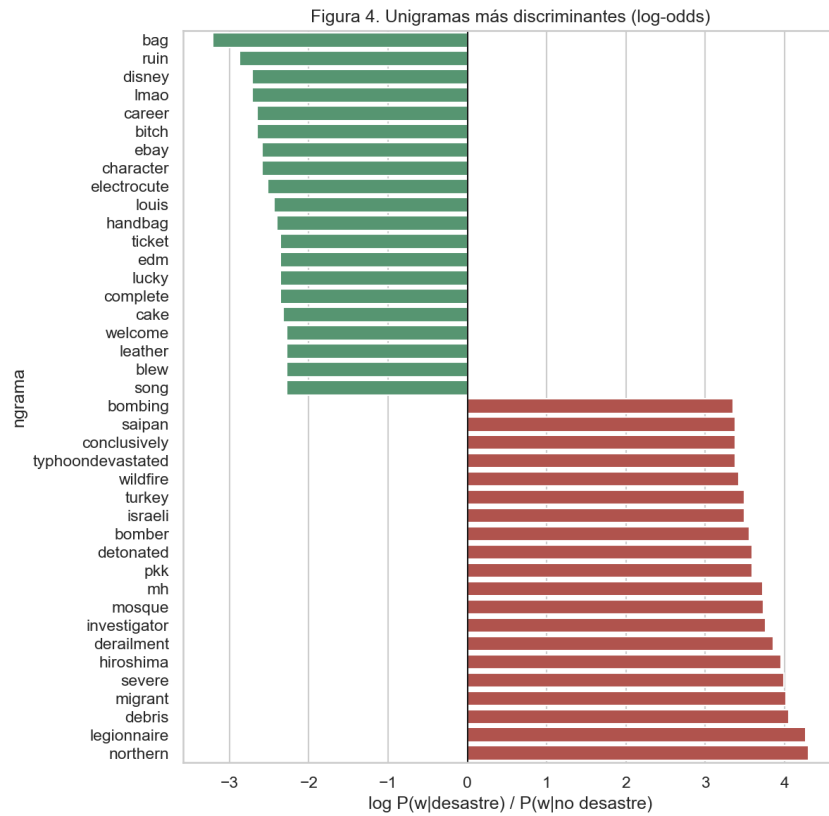


Figura 4: Unigramas más discriminantes basados en Log-odds.

Algo súper interesante fue ver el efecto de los bigramas. Si tomas la palabra *fire* sola, es muy ambigua (alguien puede decir "my mixtape is fire"). Pero si usas bigramas, el contexto se aclara: *wild fire* casi siempre es un desastre, mientras que *fire truck* no nos dice mucho. Los trigramas, por el contrario, aparecían tan poco que decidimos no meterlos al modelo para no complicarlo por gusto.

La columna **keyword** también fue clave. Palabras como *derailment* o *wreckage* nos daban un 100 % de seguridad de que era un desastre, así que pegamos esta columna al texto.

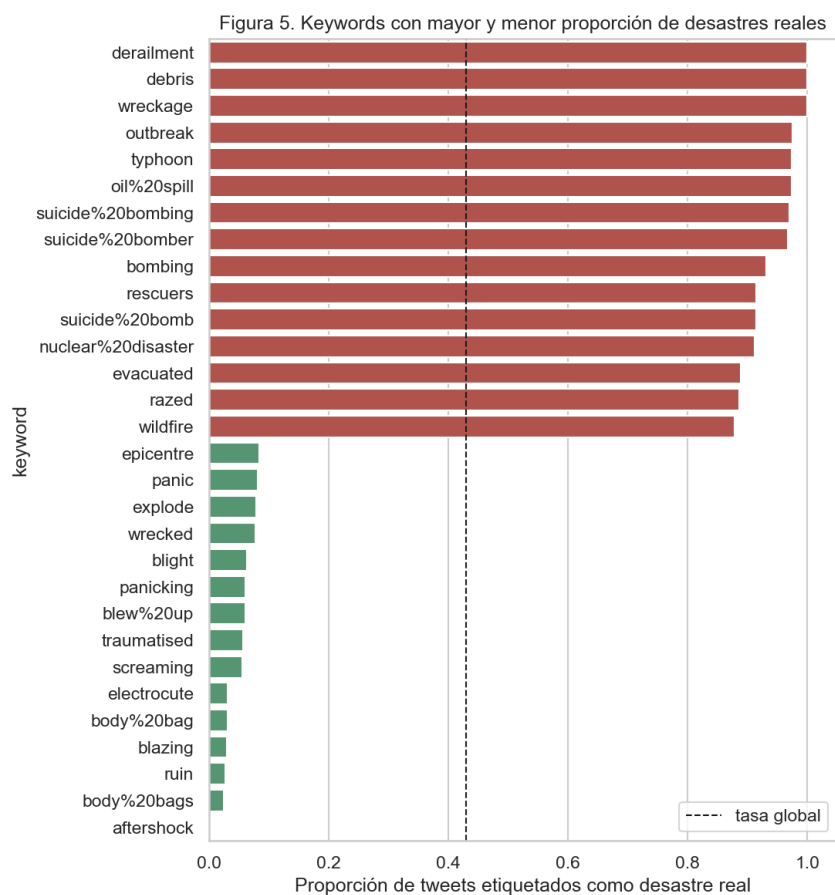


Figura 5: Keywords con mayor y menor proporción de desastres reales.

Aquí dejamos algunas nubes de palabras y gráficos para visualizar mejor qué vocabulario se usó.

Figura 6. Nubes de palabras por categoría

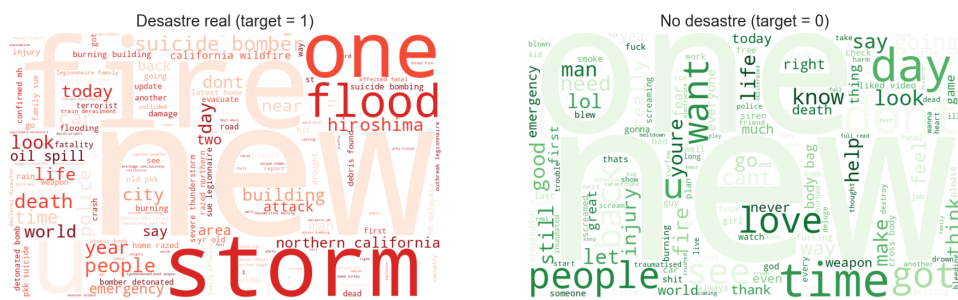


Figura 6: Nubes de palabras divididas por categoría.

Figura 8. 25 palabras más frecuentes, desagregadas por categoría

Palabra	Desastre	No desastre	Total
fire	265	85	350
like	90	255	345
not	90	210	300
get	70	185	255
no	110	145	255
new	55	170	225
one	70	135	205
news	145	60	205
people	105	95	200
dont	50	140	190
video	70	105	175
time	70	105	175
disaster	120	40	160
emergency	75	80	155
body	35	115	150
year	85	65	150
day	45	95	140
building	95	45	140
police	105	35	140
home	100	35	135
family	105	30	135
would	35	95	130
go	40	90	130
say	65	65	130
still	55	75	130

Notamos que algunas palabras que suenan muy feas, como *emergency* o *weapon*, en realidad se usan casi igual en ambos tipos de tweets (la gente es muy exagerada en internet). Lo bueno es que TF-IDF se encargó de bajarle la importancia a esas palabras automáticamente.

4. Análisis de Sentimiento

Para ver el tono emocional de los tweets, usamos el analizador VADER de NLTK. Como VADER se guía mucho por la puntuación, las mayúsculas y las caritas, le pasamos una versión del texto que no estaba limpiada al 100 % para no perder esos detalles.

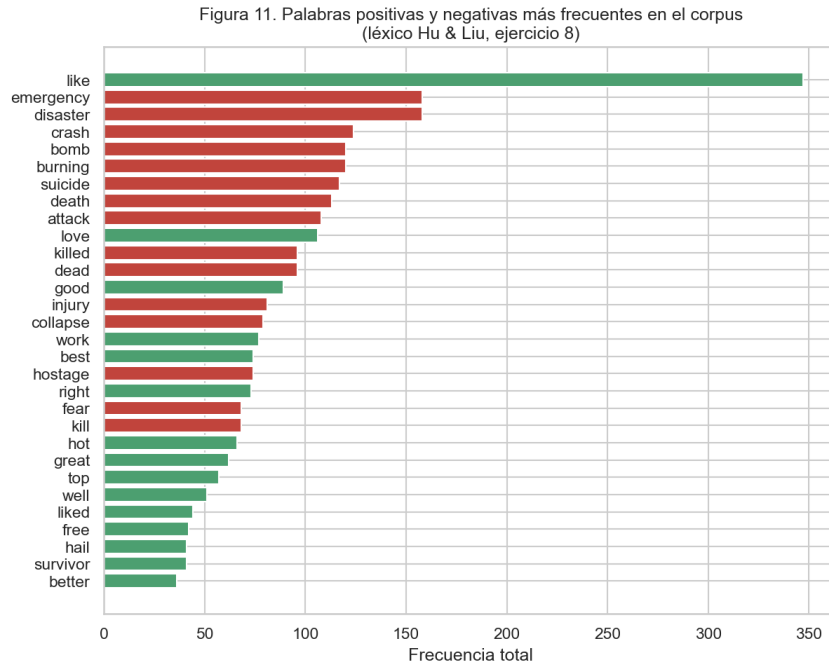


Figura 9: Palabras positivas y negativas más frecuentes en el corpus.

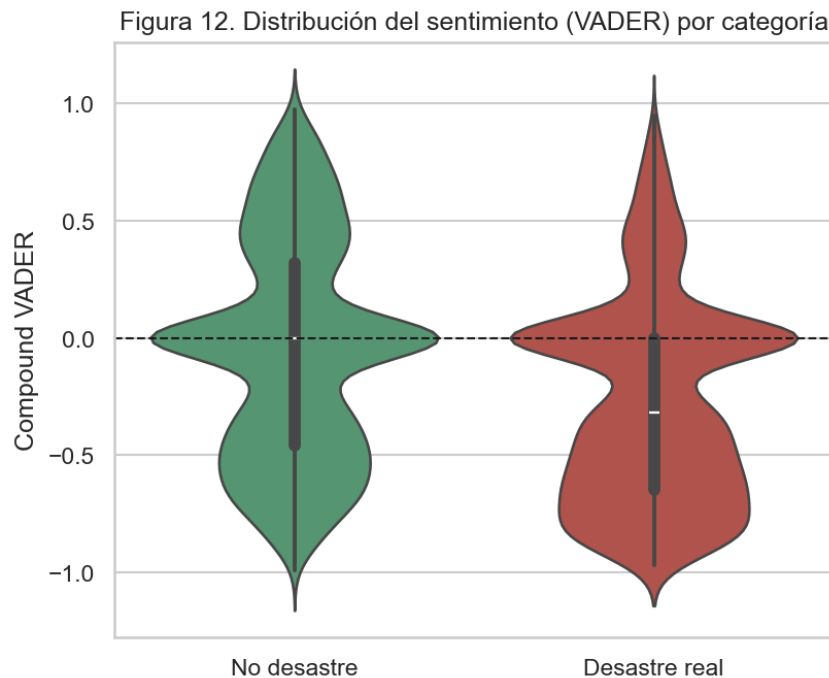


Figura 10: Distribución del sentimiento (VADER) por categoría.

Al ver los resultados, notamos que VADER a veces se confunde con el sarcasmo o con la gente que bromea usando lenguaje violento, ya que lo toma de forma muy literal.

5. Modelos de Clasificación y la Variable de Negatividad

Armamos nuestra matriz con TF-IDF incluyendo unigramas y bigramas, y probamos algoritmos como Naive Bayes, Regresión Logística, SVM Lineal y Random Forest. Los modelos lineales (como el SVM y la Regresión) fueron los que mejor se portaron.

Figura 9. Desempeño de los modelos de clasificación

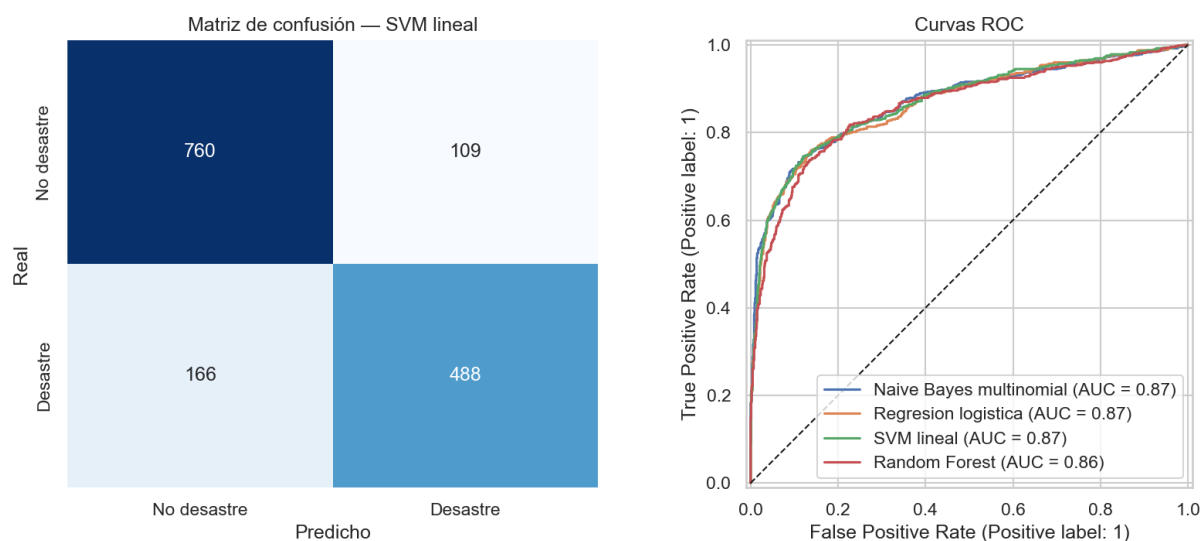


Figura 11: Desempeño preliminar de los modelos de clasificación.

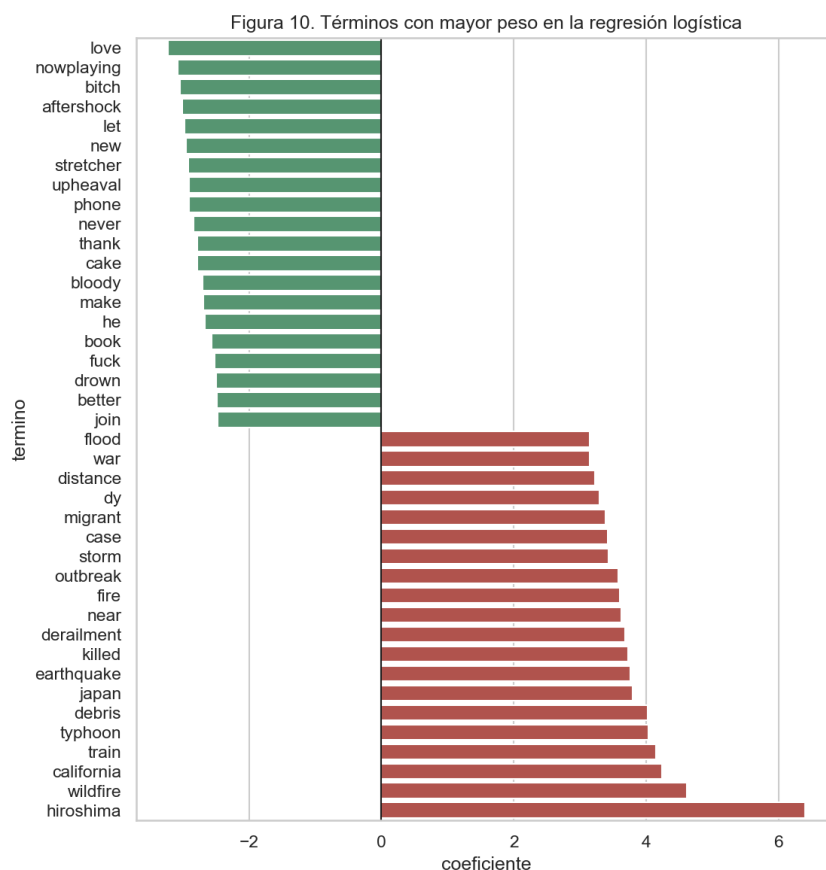


Figura 12: Términos con mayor peso en la regresión logística.

Después, en el ejercicio 10, quisimos ver si pasándole al modelo qué tan "negativo" era el tweet (según VADER) lográbamos subir las métricas. Extraímos ese puntaje y se lo pegamos a los datos.

Sorprendentemente, al volver a entrenar la Regresión Logística, los resultados fueron exactamente los mismos: 82 % de exactitud y 77 % de F1-score.

Creemos que esto pasa porque la matriz de TF-IDF ya hace un trabajo excelente identificando palabras clave (como *bomb* o *death*) y los bigramas ya le dan el contexto. El puntaje de VADER se basa en esas mismas palabras, así que terminamos pasándole al modelo información que en el fondo ya sabía, por lo que no hubo ninguna mejora.

6. Función de Clasificación Práctica

Como cierre del laboratorio, armamos una función en Python llamada `clasificar_tweet`. La idea es que cualquiera pueda meter un texto crudo de Twitter y la función se encarga de todo: lo limpia, calcula su negatividad, lo vectoriza y usa el modelo final para predecir si es un desastre o no. Con esto demostramos que todo el flujo que construimos es utilizable en la vida real.

7. Referencias

- Hutto, C.J. & Gilbert, E.E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Eighth International Conference on Weblogs and Social Media (ICWSM-14). Ann Arbor, MI, June 2014.
- Jurafsky, D., & Martin, J. H. (2014). N-Grams. Speech and Language Processing.
- Kaggle. Natural Language Processing with Disaster Tweets.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.